

Intelligent Supply Chain Decision System

Project Description:

Design and implement a **database-driven decision support system** that manages and optimizes a **multi-level supply chain network**.

The system must model **suppliers, warehouses, retailers, shipments, and orders**, while dynamically determining **optimal paths, stock allocations, and restocking strategies** — purely using **SQL logic and database design**, not application code.

This project requires students to use **recursive queries, aggregate analysis, trigger-based reasoning, and constraint-based logic** to simulate real-world decision-making.

Scenario:

A company manages a three-tier supply network:

- **Suppliers** deliver raw materials to **warehouses**.
- **Warehouses** distribute products to **retailers**.
- **Retailers** serve **customers** and place restocking orders.

Each order travels through multiple hops. Students must build a database that can:

1. Track inventory levels at every node.
2. Record shipments and their paths.
3. Automatically detect supply shortages and generate restock suggestions.
4. Compute the **shortest-cost route** between supplier and retailer (using recursive SQL or advanced join logic).
5. Prioritize urgent orders based on deadlines and available stock.

Core Functional Requirements

1. Database Design (High-Level Logic)

- Entities:
Supplier, Warehouse, Retailer, Product, Inventory, Shipment, Order, Route, Restock_Request.
- Relationships:
 - Suppliers ↔ Warehouses (many-to-many via shipments)
 - Warehouses ↔ Retailers (many-to-many)
 - Products exist at multiple levels.
- Design must handle recursive “path” relationships (routes with multiple legs).

2. Complex Logic Requirements

Students must **express reasoning in SQL**, not in procedural code.

Examples:

- Detect all retailers that cannot fulfill orders due to low inventory.
- Find **alternative fulfillment paths** (e.g., another warehouse can serve that retailer) using recursive CTEs.
- Automatically trigger a **restock request** when a product’s quantity drops below a threshold.
- Identify **bottleneck nodes** — warehouses through which the highest number of shipments flow.
- Compute **cumulative transportation cost** for each product from supplier → warehouse → retailer.

3. Decision-Support Queries

- “What is the cheapest available route from Supplier A to Retailer R for Product X?”
- “Which warehouses are at risk of stockout within the next 7 days based on order frequency?”
- “If Warehouse W shuts down, which retailers are impacted and what are their fallback routes?”
- “Which suppliers contribute to 80% of total shipping volume (Pareto analysis)?”

4. Advanced SQL Requirements

- Recursive CTEs for multi-hop routing (transitive closure).
- Window functions for ranking suppliers or bottlenecks.
- Triggers for restock automation.
- Transaction handling for order fulfillment.
- Indexing and optimization for route lookup and stock queries.

5. Analytical Challenge

- Implement a **what-if analysis** query:
“If supplier S1 is delayed by 3 days, what orders will miss their deadlines?”
- Simulate this entirely using SQL temporal logic (no external scripts).